

AI_PROMPTS.md — Ethara Seat Allocation System

This project was built with AI assistance. Below are the prompts used at each stage, what the AI produced correctly, what it got wrong, what was fixed manually, and how each was verified.

Companion docs: [README](#) · [DATABASE SCHEMA](#) · [API DOCUMENTATION](#) · [DEBUGGING NOTES](#) · [DEPLOYMENT NOTES](#)

Prompt 1 – Architecture / Planning

Prompt used: "Design the architecture for a FastAPI + React seat allocation system for ~5,000 employees, covering employees, projects, seats, seat allocations, a dashboard, and an AI assistant. Use a layered backend (models → repositories → services → API) and a component-based React front end."

- **Correct:** Clean layered backend, sensible entity split.
- **Incorrect:** Leaned toward extra abstraction not needed at this scope.
- **Fixed:** Collapsed to single-file services/repositories per resource.
- **Verified by:** Cross-checked against the assessment's required features and endpoints.

Prompt 2 – Database Design

Prompt used: "Model the tables with SQLAlchemy 2.0 and enforce 'one active seat per employee' and 'one active employee per seat' at the DB level."

- **Correct:** Table structure, relationships, partial-index approach for the one-active-seat rule.
- **Incorrect:** Bay stored as "Bay1" → later produced a "Bay Bay1" double-label in the UI.
- **Fixed:** Normalised the bay label at display time across UI + AI output.
- **Verified by:** `alembic / create_all`, inspected tables with `psql \d`, seeded and queried counts.
- See [DATABASE SCHEMA.md](#).

Prompt 3 – Backend APIs

Prompt used: "Implement the required REST endpoints for employees, projects, seats (allocate/release/available), dashboard summaries, and auth, with role-based access for writes."

- **Correct:** All endpoints, pagination, filtering, JWT auth, role guards, typed exceptions.
- **Incorrect:** AI router never registered; wrong schema import; list page-size caps too low.
- **Fixed:** Registered the router, corrected the import, raised caps.
- **Verified by:** `curl + Swagger /docs` on every endpoint. See [API DOCUMENTATION.md](#).

Prompt 4 – Seat Allocation Logic

Prompt used: "Implement seat allocation that prevents double-booking under concurrency and auto-suggests seats near a new joiner's project team (preferred floor/zone → same floor → any floor)."

- **Correct:** Fallback logic, `SELECT ... FOR UPDATE` locking, re-check under lock, partial unique indexes with `IntegrityError` rollback.
- **Incorrect:** Release endpoint expected `{seat_id}` while the schema required `employee_id`.
- **Fixed:** Aligned the release body to `{employee_id}`.
- **Verified by:** `pytest tests/test_allocation.py` + manual allocate/release.

Prompt 5 – AI Assistant

Prompt used: "Build a rule-based natural-language assistant (no external API key) that answers: where an employee sits, project assignment, available seats by floor, project occupancy, nearby colleagues, and summaries."

- **Correct:** Intent-based fallback parser working with zero external dependency.
- **Incorrect:** Loose name matching; occupancy required the word "project"; suggested chips used names not in the seeded data.
- **Fixed:** Rewrote the parser (stopword-filtered, token-scored matching; project detection); added `GET /ai/examples` built from real data.
- **Verified by:** curl across all intents against live data + clicking chips in the UI.

Prompt 6 – Frontend

Prompt used: "Implement the provided design handoff (Ethara Seat Allocation) in the existing React 19 + Vite + Tailwind v4 + Zustand stack — same components/services, no new core libraries — wired to the real backend."

- **Correct:** Dark console shell, dashboard, employee table + detail slide-over, project cards, seat grid + allocate modal, new-joiner flow, search, AI chat — all wired to real API services.
- **Incorrect:** `ToastProvider` never mounted (blank pages); base CSS written unlayered so it overrode Tailwind text-color utilities (invisible nav/heading text).
- **Fixed:** Mounted `ToastProvider` + `ToastContainer`; wrapped base styles in `@layer base`; added explicit `text-white` on dark panels.
- **Verified by:** Real-browser walkthrough + console inspection.

Prompt 7 – Frontend ↔ Backend contract

Prompt used: "Adversarially verify every field the frontend reads against the actual FastAPI response schemas and fix mismatches."

- **Correct:** Base paths and list-wrapper handling.
- **Incorrect:** Enum casing, dashboard key names, a double-nested `/ai/query` body.
- **Fixed:** Normalised casing, corrected field names, flattened the body, enriched employee/project responses.
- **Verified by:** curl each endpoint; compared JSON keys to the components. See [DEBUGGING NOTES.md](#).

Prompt 8 – Debugging

Prompt used: "The app isn't functioning and the UI looks different from the design — diagnose and fix."

Root causes found and fixed: missing ToastProvider, page-size caps, unregistered AI router + wrong import, unlayered base CSS, weak AI parser + fake prompt names, login autofill/hint, empty migration on deploy. Full write-up in [DEBUGGING NOTES.md](#).

Prompt 9 – Deployment

Prompt used: "Deploy the frontend to Vercel and the backend + PostgreSQL to Render, wire the environment, and verify the live URLs end-to-end."

- **Correct:** Vercel CLI deploy of the static Vite build; a Render Blueprint (`render.yaml`) that provisions the web service + managed Postgres and injects env vars.
- **Incorrect:** (a) Railway hit its free-plan provision limit; (b) the empty Alembic migration created no tables on the fresh Render DB, so every query 500'd; (c) the Render free tier has no Shell to seed manually; (d) `postgres://` URL scheme needed normalizing for SQLAlchemy.
- **Fixed:** Added `scripts/bootstrap.py` (create tables + seed only if empty) to the start command so the service self-heals on first boot; normalized the DB URL in `config.py` ; set `VITE_API_BASE_URL` and `CORS_ORIGINS` to the live URLs.
- **Verified by:** curl end-to-end on the live URLs (health, login+JWT, employees=5000, dashboard, AI) and confirmed the deployed frontend bundle points to the Render API with CORS allowed. See [DEPLOYMENT NOTES.md](#).

Live: Frontend <https://frontend-lake-one-62.vercel.app> · Backend <https://ethara-backend-9kbg.onrender.com> (`/docs`).

Prompt 10 – Refactoring / Polish

Prompt used: "Clean up duplication and tighten fidelity to the design."

- **Correct:** Shared Tailwind theme tokens drive the whole UI.
- **Incorrect / noted:** Avatar/status palette duplicated across components; a few orphaned components remain.
- **Fixed:** Consolidated status/bay handling; remaining dedup tracked as low-risk cleanup.
- **Verified by:** `npm run build` (clean) + `eslint` (no new errors).

Summary of validation

- Backend: `pytest` , Swagger `/docs` , curl smoke tests on every endpoint (local + live).
- Frontend: real-browser walkthrough of every screen + console/network inspection.
- Data: confirmed API totals match Postgres row counts (nothing hardcoded).